VIT-AP UNIVERSITY, ANDHRA PRADESH

Lab Sheet 6: MongoDB Basic commands

Branch/ Class: B.Tech/M.Tech **Date:** 11-03-2026

Faculty Name: Prof. S.Gopikrishnan **School:** SCOPE

Student name: L.Yasaswini **Reg. no.:** 23BCE8928

---

1. Use MongoDB to implement the following DB operations

1. Create a database called 'vehicles' and *write* a MongoDB query to select database as "vehicles".

```
test> use vehicles
switched to db vehicles
```

2. Write a MongoDB query to display all the databases.

```
vehicles> show dbs
Exp1      16.00 KiB
admin     40.00 KiB
config   108.00 KiB
local     40.00 KiB
```

3. Create a collection called 'two_wheelers'. (use capping) and Create a collection called 'four_wheelers'.

```
vehicles> db.createCollection("two_wheelers",{capped:true,size:100000})
{ ok: 1 }
vehicles> db.createCollection("four_wheelers")
{ ok: 1 }
```

4. Add 5 two-wheeler details to the collection named 'two_wheelers'. Each document consists of following fields as bike_name, model (gear or gearless), category (100cc, 125cc, 150cc, 200cc), colors_available (red, black, blue, sport red etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
vehicles> db.two_wheelers.insertMany([
| {
| bike_name:"Activa",
| model:"gearless",
| category:"125cc",
| colors_available:["red","black","blue"],
| manufacturer:"Honda",
| performance:8,
| timestamp:new Date("2021-05-10"),
| price:80000
| },
| {
| bike_name:"Pulsar",
| model:"gear",
| category:"150cc",
| colors_available:["black","blue"],
| manufacturer:"Bajaj",
| performance:9,
| timestamp:new Date("2020-03-15"),
| price:120000
| },
| {
| bike_name:"Apache",
| model:"gear",
| category:"200cc",
| colors_available:["sport red","black"],
| manufacturer:"TVS",
| performance:8,
| timestamp:new Date("2022-01-10"),
| price:140000
| },
| {
| bike_name:"Shine",
| model:"gear",
| category:"125cc",
| colors_available:["red","black"],
| manufacturer:"Honda",
| performance:7,
| timestamp:new Date("2019-07-20"),
| price:90000
| },
```

```
| {
| bike_name:"Jupiter",
| model:"gearless",
| category:"100cc",
| colors_available:["blue","grey"],
| manufacturer:"TVS",
| performance:7,
| timestamp:new Date("2023-02-12"),
| price:85000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b183df61226cf47f7c2907'),
    '1': ObjectId('69b183df61226cf47f7c2908'),
    '2': ObjectId('69b183df61226cf47f7c2909'),
    '3': ObjectId('69b183df61226cf47f7c290a'),
    '4': ObjectId('69b183df61226cf47f7c290b')
  }
}
```

5. Add 5 four-wheeler details to the collection named ‘four_wheelers’. Each document consists of following fields as vehicle_name, model (commercial or own), category (car, lorry, bus, mini truck, heavy truck, containers), variants (vxi, zxi, petrol, diesel etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
vehicles> db.four_wheelers.insertMany([
| {
| vehicle_name:"Swift",
| model:"own",
| category:"car",
| variants:["petrol","vxi"],
| manufacturer:"Maruti",
| performance:8,
| timestamp:new Date("2021-06-12"),
| price:700000
| },
| {
| vehicle_name:"Innova",
| model:"commercial",
| category:"car",
| variants:["diesel","zxi"],
| manufacturer:"Toyota",
| performance:9,
| timestamp:new Date("2020-05-11"),
| price:1800000
| },
| {
| vehicle_name:"Ashok Leyland Bus",
| model:"commercial",
| category:"bus",
| variants:["diesel"],
| manufacturer:"Ashok Leyland",
| performance:7,
| timestamp:new Date("2019-03-20"),
| price:2500000
| },
| {
| vehicle_name:"Mahindra Truck",
| model:"commercial",
| category:"heavy truck",
| variants:["diesel"],
| manufacturer:"Mahindra",
| performance:8,
| timestamp:new Date("2018-09-12"),
| price:3000000
```

```
| },
| {
| vehicle_name:"Eeco",
| model:"own",
| category:"mini truck",
| variants:["petrol"],
| manufacturer:"Maruti",
| performance:6,
| timestamp:new Date("2022-04-18"),
| price:600000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b183e961226cf47f7c290c'),
    '1': ObjectId('69b183e961226cf47f7c290d'),
    '2': ObjectId('69b183e961226cf47f7c290e'),
    '3': ObjectId('69b183e961226cf47f7c290f'),
    '4': ObjectId('69b183e961226cf47f7c2910')
  }
}
```

6. Write a MongoDB query to display all documents available in two_wheelers and four_wheelers.

```
vehicles> db.two_wheelers.find()
[
  {
    _id: ObjectId('69b183df61226cf47f7c2907'),
    bike_name: 'Activa',
    model: 'gearless',
    category: '125cc',
    colors_available: [ 'red', 'black', 'blue' ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: ISODate('2021-05-10T00:00:00.000Z'),
    price: 80000
  },
  {
    _id: ObjectId('69b183df61226cf47f7c2908'),
    bike_name: 'Pulsar',
    model: 'gear',
    category: '150cc',
    colors_available: [ 'black', 'blue' ],
    manufacturer: 'Bajaj',
    performance: 9,
    timestamp: ISODate('2020-03-15T00:00:00.000Z'),
    price: 120000
  },
  {
    _id: ObjectId('69b183df61226cf47f7c2909'),
    bike_name: 'Apache',
    model: 'gear',
    category: '200cc',
    colors_available: [ 'sport red', 'black' ],
    manufacturer: 'TVS',
    performance: 8,
    timestamp: ISODate('2022-01-10T00:00:00.000Z'),
    price: 140000
  },
  {
    _id: ObjectId('69b183df61226cf47f7c290a'),
    bike_name: 'Shine',
    model: 'gear',
    category: '125cc',
```

```
    colors_available: [ 'red', 'black' ],
    manufacturer: 'Honda',
    performance: 7,
    timestamp: ISODate('2019-07-20T00:00:00.000Z'),
    price: 90000
  },
  {
    _id: ObjectId('69b183df61226cf47f7c290b'),
    bike_name: 'Jupiter',
    model: 'gearless',
    category: '100cc',
    colors_available: [ 'blue', 'grey' ],
    manufacturer: 'TVS',
    performance: 7,
    timestamp: ISODate('2023-02-12T00:00:00.000Z'),
    price: 85000
  }
]
```

```
vehicles> db.four_wheelers.find()
[
  {
    _id: ObjectId('69b183e961226cf47f7c290c'),
    vehicle_name: 'Swift',
    model: 'own',
    category: 'car',
    variants: [ 'petrol', 'vxi' ],
    manufacturer: 'Maruti',
    performance: 8,
    timestamp: ISODate('2021-06-12T00:00:00.000Z'),
    price: 700000
  },
  {
    _id: ObjectId('69b183e961226cf47f7c290d'),
    vehicle_name: 'Innova',
    model: 'commercial',
    category: 'car',
    variants: [ 'diesel', 'zxi' ],
    manufacturer: 'Toyota',
    performance: 9,
    timestamp: ISODate('2020-05-11T00:00:00.000Z'),
    price: 1800000
  },
  {
    _id: ObjectId('69b183e961226cf47f7c290e'),
    vehicle_name: 'Ashok Leyland Bus',
    model: 'commercial',
    category: 'bus',
    variants: [ 'diesel' ],
    manufacturer: 'Ashok Leyland',
    performance: 7,
    timestamp: ISODate('2019-03-20T00:00:00.000Z'),
    price: 2500000
  },
  {
    _id: ObjectId('69b183e961226cf47f7c290f'),
    vehicle_name: 'Mahindra Truck',
    model: 'commercial',
    category: 'heavy truck',
```

```
    variants: [ 'diesel' ],
    manufacturer: 'Mahindra',
    performance: 8,
    timestamp: ISODate('2018-09-12T00:00:00.000Z'),
    price: 3000000
  },
  {
    _id: ObjectId('69b183e961226cf47f7c2910'),
    vehicle_name: 'Eeco',
    model: 'own',
    category: 'mini truck',
    variants: [ 'petrol' ],
    manufacturer: 'Maruti',
    performance: 6,
    timestamp: ISODate('2022-04-18T00:00:00.000Z'),
    price: 600000
  }
]
```

7. Write a MongoDB query to display only vehicle name and price in all the collection of the database

```
vehicles> db.two_wheelers.find({}, {bike_name:1,price:1,_id:0})
[
  { bike_name: 'Activa', price: 80000 },
  { bike_name: 'Pulsar', price: 120000 },
  { bike_name: 'Apache', price: 140000 },
  { bike_name: 'Shine', price: 90000 },
  { bike_name: 'Jupiter', price: 85000 }
]
vehicles> db.four_wheelers.find({}, {vehicle_name:1,price:1,_id:0})
[
  { vehicle_name: 'Swift', price: 700000 },
  { vehicle_name: 'Innova', price: 1800000 },
  { vehicle_name: 'Ashok Leyland Bus', price: 2500000 },
  { vehicle_name: 'Mahindra Truck', price: 3000000 },
  { vehicle_name: 'Eeco', price: 600000 }
]
```

8. Write a MongoDB query to display two_wheelers from a particular company

```
vehicles> db.two_wheelers.find({manufacturer:"Honda"})
[
  {
    _id: ObjectId('69b183df61226cf47f7c2907'),
    bike_name: 'Activa',
    model: 'gearless',
    category: '125cc',
    colors_available: [ 'red', 'black', 'blue' ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: ISODate('2021-05-10T00:00:00.000Z'),
    price: 80000
  },
  {
    _id: ObjectId('69b183df61226cf47f7c290a'),
    bike_name: 'Shine',
    model: 'gear',
    category: '125cc',
    colors_available: [ 'red', 'black' ],
    manufacturer: 'Honda',
    performance: 7,
    timestamp: ISODate('2019-07-20T00:00:00.000Z'),
    price: 90000
  }
]
```

9. Write a MongoDB query to display four_wheelers available in diesel variants

```
vehicles> db.four_wheelers.find({variants:"diesel"})
[
  {
    _id: ObjectId('69b183e961226cf47f7c290d'),
    vehicle_name: 'Innova',
    model: 'commercial',
    category: 'car',
    variants: [ 'diesel', 'zxi' ],
    manufacturer: 'Toyota',
    performance: 9,
    timestamp: ISODate('2020-05-11T00:00:00.000Z'),
    price: 1800000
  },
  {
    _id: ObjectId('69b183e961226cf47f7c290e'),
    vehicle_name: 'Ashok Leyland Bus',
    model: 'commercial',
    category: 'bus',
    variants: [ 'diesel' ],
    manufacturer: 'Ashok Leyland',
    performance: 7,
    timestamp: ISODate('2019-03-20T00:00:00.000Z'),
    price: 2500000
  },
  {
    _id: ObjectId('69b183e961226cf47f7c290f'),
    vehicle_name: 'Mahindra Truck',
    model: 'commercial',
    category: 'heavy truck',
    variants: [ 'diesel' ],
    manufacturer: 'Mahindra',
    performance: 8,
    timestamp: ISODate('2018-09-12T00:00:00.000Z'),
    price: 3000000
  }
]
```

10. Write a MongoDB query to display vehicles name, category and manufacturer details whose rating is more than 5.

```
vehicles> db.two_wheelers.find({performance:{$gt:5}},{bike_name:1,category:1,manufacturer:1,_id:0})
[
  { bike_name: 'Activa', category: '125cc', manufacturer: 'Honda' },
  { bike_name: 'Pulsar', category: '150cc', manufacturer: 'Bajaj' },
  { bike_name: 'Apache', category: '200cc', manufacturer: 'TVS' },
  { bike_name: 'Shine', category: '125cc', manufacturer: 'Honda' },
  { bike_name: 'Jupiter', category: '100cc', manufacturer: 'TVS' }
]
vehicles> db.four_wheelers.find({performance:{$gt:5}},{vehicle_name:1,categocategory:1,manufacturer:1,_id:0})
[
  { vehicle_name: 'Swift', category: 'car', manufacturer: 'Maruti' },
  { vehicle_name: 'Innova', category: 'car', manufacturer: 'Toyota' },
  {
    vehicle_name: 'Ashok Leyland Bus',
    category: 'bus',
    manufacturer: 'Ashok Leyland'
  },
  {
    vehicle_name: 'Mahindra Truck',
    category: 'heavy truck',
    manufacturer: 'Mahindra'
  },
  {
    vehicle_name: 'Eeco',
    category: 'mini truck',
    manufacturer: 'Maruti'
  }
]
```

2. Use MongoDB to implement the following DB operations for a Zoo

1. Create a database called ‘animal’ and *write* a MongoDB query to select database as ‘animal’.

```
vehicles> use animal
switched to db animal
```

2. Write a MongoDB query to display all the databases.

```
switched to db animal
animal> show dbs
Exp1       16.00 KiB
admin      40.00 KiB
config    108.00 KiB
local      40.00 KiB
vehicles   80.00 KiB
```

3. Create a collection called ‘wild_animals’.(use capping) and Create a collection called ‘domestic_animals’.

```
animal> db.createCollection("wild_animals",{capped:true,size:100000})
{ ok: 1 }
animal> db.wild_animals.insertMany([
| {
| animal_name:"Lion",
| nature:"harm",
| favorite_foods:["meat","deer"],
| care_taker_name:"Ravi",
| life_span:15,
| timestamp:new Date("2020-01-10"),
| expenses:50000
| },
| {
| animal_name:"Tiger",
| nature:"harm",
| favorite_foods:["meat","rabbits"],
| care_taker_name:"Suresh",
| life_span:18,
| timestamp:new Date("2019-05-12"),
| expenses:60000
| },
| {
| animal_name:"Elephant",
| nature:"harmless",
| favorite_foods:["grass","fruits"],
| care_taker_name:"Ravi",
| life_span:60,
| timestamp:new Date("2018-03-20"),
| expenses:70000
| },
| {
| animal_name:"Bear",
| nature:"harm",
| favorite_foods:["fish","honey"],
| care_taker_name:"Mahesh",
| life_span:25,
| timestamp:new Date("2021-07-15"),
| expenses:40000
| },
| {
```

```
| animal_name:"Deer",
| nature:"harmless",
| favorite_foods:["grass","leaves"],
| care_taker_name:"Suresh",
| life_span:20,
| timestamp:new Date("2022-02-18"),
| expenses:20000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b189b961226cf47f7c2911'),
    '1': ObjectId('69b189b961226cf47f7c2912'),
    '2': ObjectId('69b189b961226cf47f7c2913'),
    '3': ObjectId('69b189b961226cf47f7c2914'),
    '4': ObjectId('69b189b961226cf47f7c2915')
  }
}
```

4. Add 5 wild_animal details to the collection named ‘wild_animals’. Each document consists of following fields as animal_name, nature (harm or harmless), favorite_foods (meat, rabbits, deer etc) as array, care_taker_name, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
animal> db.domestic_animals.insertMany([
| {
| animal_name:"Dog",
| gender:"male",
| favorite_foods:["meat","biscuits"],
| animal_petname:"Tommy",
| life_span:12,
| timestamp:new Date("2021-04-10"),
| expenses:5000
| },
| {
| animal_name:"Cat",
| gender:"female",
| favorite_foods:["fish","milk"],
| animal_petname:"Kitty",
| life_span:10,
| timestamp:new Date("2020-06-12"),
| expenses:4000
| },
| {
| animal_name:"Cow",
| gender:"female",
| favorite_foods:["grass"],
| animal_petname:"Ganga",
| life_span:20,
| timestamp:new Date("2019-09-15"),
| expenses:6000
| },
| {
| animal_name:"Goat",
| gender:"male",
| favorite_foods:["leaves"],
| animal_petname:"Chotu",
| life_span:15,
| timestamp:new Date("2022-01-05"),
| expenses:3000
| },
| {
| animal_name:"Rabbit",
```

```
| gender:"female",
| favorite_foods:["carrot","grass"],
| animal_petname:"Bunny",
| life_span:8,
| timestamp:new Date("2023-03-08"),
| expenses:2000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b189c261226cf47f7c2916'),
    '1': ObjectId('69b189c261226cf47f7c2917'),
    '2': ObjectId('69b189c261226cf47f7c2918'),
    '3': ObjectId('69b189c261226cf47f7c2919'),
    '4': ObjectId('69b189c261226cf47f7c291a')
  }
}
```

5. Add 5 domestic-animal details to the collection named ‘domestic_animals’. Each document consists of following fields as animal_name, gender (male or female), favorite_foods (meat, rabbits, deer etc) as array, animal_petname, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
animal> db.wild_animals.find()
[
  {
    _id: ObjectId('69b189b961226cf47f7c2911'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Ravi',
    life_span: 15,
    timestamp: ISODate('2020-01-10T00:00:00.000Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69b189b961226cf47f7c2912'),
    animal_name: 'Tiger',
    nature: 'harm',
    favorite_foods: [ 'meat', 'rabbits' ],
    care_taker_name: 'Suresh',
    life_span: 18,
    timestamp: ISODate('2019-05-12T00:00:00.000Z'),
    expenses: 60000
  },
  {
    _id: ObjectId('69b189b961226cf47f7c2913'),
    animal_name: 'Elephant',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'fruits' ],
    care_taker_name: 'Ravi',
    life_span: 60,
    timestamp: ISODate('2018-03-20T00:00:00.000Z'),
    expenses: 70000
  },
  {
    _id: ObjectId('69b189b961226cf47f7c2914'),
    animal_name: 'Bear',
    nature: 'harm',
    favorite_foods: [ 'fish', 'honey' ],
    care_taker_name: 'Mahesh',
    life_span: 25,
    timestamp: ISODate('2021-07-15T00:00:00.000Z'),
    expenses: 40000
```

```
  },
  {
    _id: ObjectId('69b189b961226cf47f7c2915'),
    animal_name: 'Deer',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'leaves' ],
    care_taker_name: 'Suresh',
    life_span: 20,
    timestamp: ISODate('2022-02-18T00:00:00.000Z'),
    expenses: 20000
  }
]
```

6. Write a MongoDB query to display all documents available in wild_animals and domastic_animals.

```
animal> db.wild_animals.find()
[
  {
    _id: ObjectId('69b189b961226cf47f7c2911'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Ravi',
    life_span: 15,
    timestamp: ISODate('2020-01-10T00:00:00.000Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69b189b961226cf47f7c2912'),
    animal_name: 'Tiger',
    nature: 'harm',
    favorite_foods: [ 'meat', 'rabbits' ],
    care_taker_name: 'Suresh',
    life_span: 18,
    timestamp: ISODate('2019-05-12T00:00:00.000Z'),
    expenses: 60000
  },
  {
    _id: ObjectId('69b189b961226cf47f7c2913'),
    animal_name: 'Elephant',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'fruits' ],
    care_taker_name: 'Ravi',
    life_span: 60,
    timestamp: ISODate('2018-03-20T00:00:00.000Z'),
    expenses: 70000
  },
  {
    _id: ObjectId('69b189b961226cf47f7c2914'),
    animal_name: 'Bear',
    nature: 'harm',
    favorite_foods: [ 'fish', 'honey' ],
    care_taker_name: 'Mahesh',
    life_span: 25,
    timestamp: ISODate('2021-07-15T00:00:00.000Z'),
```

```
    expenses: 40000
  },
  {
    _id: ObjectId('69b189b961226cf47f7c2915'),
    animal_name: 'Deer',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'leaves' ],
    care_taker_name: 'Suresh',
    life_span: 20,
    timestamp: ISODate('2022-02-18T00:00:00.000Z'),
    expenses: 20000
  }
]
```

```
animal> db.wild_animals.find().pretty()
[
  {
    _id: ObjectId('69b189b961226cf47f7c2911'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Ravi',
    life_span: 15,
    timestamp: ISODate('2020-01-10T00:00:00.000Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69b189b961226cf47f7c2912'),
    animal_name: 'Tiger',
    nature: 'harm',
    favorite_foods: [ 'meat', 'rabbits' ],
    care_taker_name: 'Suresh',
    life_span: 18,
    timestamp: ISODate('2019-05-12T00:00:00.000Z'),
    expenses: 60000
  },
  {
    _id: ObjectId('69b189b961226cf47f7c2913'),
    animal_name: 'Elephant',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'fruits' ],
    care_taker_name: 'Ravi',
    life_span: 60,
    timestamp: ISODate('2018-03-20T00:00:00.000Z'),
    expenses: 70000
  },
  {
    _id: ObjectId('69b189b961226cf47f7c2914'),
    animal_name: 'Bear',
    nature: 'harm',
    favorite_foods: [ 'fish', 'honey' ],
    care_taker_name: 'Mahesh',
    life_span: 25,
    timestamp: ISODate('2021-07-15T00:00:00.000Z'),
```

```
    expenses: 40000
  },
  {
    _id: ObjectId('69b189b961226cf47f7c2915'),
    animal_name: 'Deer',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'leaves' ],
    care_taker_name: 'Suresh',
    life_span: 20,
    timestamp: ISODate('2022-02-18T00:00:00.000Z'),
    expenses: 20000
  }
]
```

7. Write a MongoDB query to display only animal name and expenses in all the collection of the database

```
animal> db.wild_animals.find({}, {animal_name:1,expenses:1,_id:0})
[
  { animal_name: 'Lion', expenses: 50000 },
  { animal_name: 'Tiger', expenses: 60000 },
  { animal_name: 'Elephant', expenses: 70000 },
  { animal_name: 'Bear', expenses: 40000 },
  { animal_name: 'Deer', expenses: 20000 }
]
animal> db.domestic_animals.find({}, {animal_name:1,expenses:1,_id:0})
[
  { animal_name: 'Dog', expenses: 5000 },
  { animal_name: 'Cat', expenses: 4000 },
  { animal_name: 'Cow', expenses: 6000 },
  { animal_name: 'Goat', expenses: 3000 },
  { animal_name: 'Rabbit', expenses: 2000 }
]
```

8. Write a MongoDB query to display domestic_animals whose life is a particular year

```
animal> db.domestic_animals.find({life_span:10})
[
  {
    _id: ObjectId('69b189c261226cf47f7c2917'),
    animal_name: 'Cat',
    gender: 'female',
    favorite_foods: [ 'fish', 'milk' ],
    animal_petname: 'Kitty',
    life_span: 10,
    timestamp: ISODate('2020-06-12T00:00:00.000Z'),
    expenses: 4000
  }
]
```

9. Write a MongoDB query to display wild_animals available under a particular care_taker

```
animal> db.wild_animals.find({care_taker_name:"Ravi"})
[
  {
    _id: ObjectId('69b189b961226cf47f7c2911'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Ravi',
    life_span: 15,
    timestamp: ISODate('2020-01-10T00:00:00.000Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69b189b961226cf47f7c2913'),
    animal_name: 'Elephant',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'fruits' ],
    care_taker_name: 'Ravi',
    life_span: 60,
    timestamp: ISODate('2018-03-20T00:00:00.000Z'),
    expenses: 70000
  }
]
```

10. Write a MongoDB query to display animal name, favorite_foods and expenses details whose lifespan is more than 5 years.

```
animal> db.wild_animals.find(
| {life_span:{$gt:5}},
| {animal_name:1,favorite_foods:1,expenses:1,_id:0}
| )
[
  {
    animal_name: 'Lion',
    favorite_foods: [ 'meat', 'deer' ],
    expenses: 50000
  },
  {
    animal_name: 'Tiger',
    favorite_foods: [ 'meat', 'rabbits' ],
    expenses: 60000
  },
  {
    animal_name: 'Elephant',
    favorite_foods: [ 'grass', 'fruits' ],
    expenses: 70000
  },
  {
    animal_name: 'Bear',
    favorite_foods: [ 'fish', 'honey' ],
    expenses: 40000
  },
  {
    animal_name: 'Deer',
    favorite_foods: [ 'grass', 'leaves' ],
    expenses: 20000
  }
]
```

```
animal> db.domestic_animals.find(
| {life_span:{$gt:5}},
| {animal_name:1,favorite_foods:1,expenses:1,_id:0}
| )
[
  {
    animal_name: 'Dog',
    favorite_foods: [ 'meat', 'biscuits' ],
    expenses: 5000
  },
  {
    animal_name: 'Cat',
    favorite_foods: [ 'fish', 'milk' ],
    expenses: 4000
  },
  { animal_name: 'Cow', favorite_foods: [ 'grass' ], expenses: 6000 },
  { animal_name: 'Goat', favorite_foods: [ 'leaves' ], expenses: 3000 },
  {
    animal_name: 'Rabbit',
    favorite_foods: [ 'carrot', 'grass' ],
    expenses: 2000
  }
]
```